Additional Components and Considerations

A notification system involves much more than simply sending and receiving notifications.

In large-scale systems, several additional components and considerations are required to improve:

- Reliability
- Scalability
- User experience
- Security
- Monitoring
- Analytics

The important additional components are:

- Notification templates
- Notification settings
- Rate limiting
- Retry mechanism
- Security in push notifications
- Monitoring queued notifications
- Event tracking

1. Notification Template

Large-scale notification systems send:

- Millions of notifications daily

Many notifications follow:

- Similar layouts
- Similar formats
- Similar structures

To avoid creating every notification from scratch:

Notification Templates are introduced

What is a Notification Template?

A notification template is:

```
A preformatted notification structure used to generate notifications dynamically.
```

Templates contain:

- Formatting
- Styling
- Placeholders
- Tracking links
- Reusable content

Example Push Notification Template

BODY

```
You dreamed of it. We dared it. [ITEM NAME] is back – only until [DATE].
```

CTA (Call To Action)

```
None

Order Now. Or, Save My [ITEM NAME]
```

Dynamic Parameters

Templates contain placeholders such as:

- `[ITEM NAME]`
- `[DATE]`

These placeholders are replaced dynamically at runtime.

Benefits of Notification Templates

1. Consistent Notification Format

All notifications follow the same design structure.

2. Reduce Human Errors

Avoids repeatedly writing notifications manually.

3. Save Development Time

Reusable templates simplify notification creation.

4. Easier Maintenance

Updating one template updates all related notifications.

2. Notification Settings

Users may receive:

- Too many push notifications
- Too many emails
- Too many SMS messages

This can overwhelm users.

Thus, users are given:

Fine-grained Notification Control

Notification Settings Table

The notification setting information is stored in a table.

Fields

Field	Type	Description
user_id	bigInt	Unique user ID

channel	varchar	Push, Email, SMS
opt_in	boolean	Whether notifications are enabled

Purpose

Allows users to:

- Enable notifications
- Disable notifications
- Choose preferred channels

Notification Sending Check

Before sending any notification:

The system checks:

```
None
Whether the user has opted-in for that notification type
```

Example

If:

- User disables SMS notifications

Then:

- SMS messages are not sent

Benefits

- Better user experience
- Prevents notification fatigue
- Reduces spam complaints

3. Rate Limiting

Users should not receive excessive notifications in a short time.

Too many notifications can cause users to:

- Disable notifications
- Uninstall apps
- Ignore alerts

Thus:

Rate limiting is introduced

Purpose of Rate Limiting

Limit the number of notifications sent to a user within a time window.

Example Rules

- Maximum 5 push notifications per hour
- Maximum 2 promotional emails per day
- Maximum 3 SMS alerts per hour

Benefits

- Prevents overwhelming users
- Improves user engagement
- Reduces spam behavior

4. Retry Mechanism

Third-party services can fail temporarily.

Examples:

- APNS downtime
- FCM timeout
- SMS gateway failure
- Email service issue

To handle failures:

Retry mechanism is implemented

Retry Flow

1. Notification delivery fails
2. Notification is added back to message queue
3. Workers retry delivery later

Persistent Failures

If retries continue failing:

- Alerts are sent to developers

Purpose:

- Manual investigation
- Faster incident response

Benefits

- Improves reliability
- Handles temporary outages
- Reduces notification loss

5. Security in Push Notifications

Push notification APIs must be secured.

Otherwise:

- Attackers may send fake notifications
- Spam notifications may be generated

Security Mechanism

For iOS and Android:

- `appKey`
- `appSecret`

are used for authentication.

Purpose

Ensure that:

```
Only authenticated and verified clients can send notifications.
```

Benefits

- Prevents unauthorized access
- Protects push notification APIs
- Improves system security

6. Monitor Queued Notifications

Queue monitoring is extremely important.

One critical metric is:

Number of queued notifications

Why Queue Monitoring Matters?

If queue size becomes very large:

- Workers are not processing events fast enough

This causes:

- Notification delays
- Increased latency
- Poor user experience

Solution

Scale workers horizontally.

Add:

- More worker servers
- More processing capacity

Figure Reference

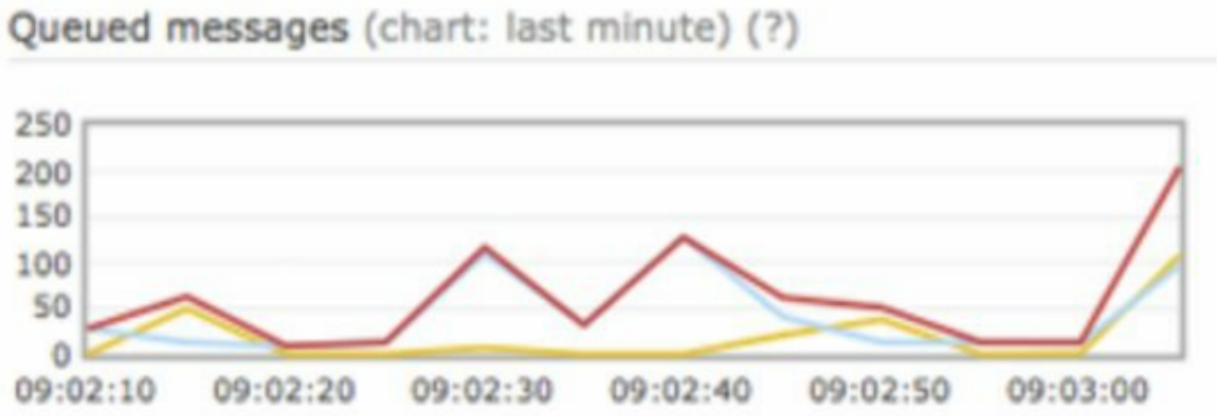

Figure shows:

- Example graph of queued messages waiting for processing

Benefits of Queue Monitoring

- Detect system bottlenecks
- Prevent delivery delays
- Improve throughput

7. Event Tracking

Notification analytics are important for understanding:

- User behavior
- Engagement
- Notification effectiveness

Metrics Tracked

Examples include:

- Open rate
- Click rate
- Engagement rate
- Conversion rate

Analytics Service

An analytics service tracks notification events.

Integration between:

- Notification system
- Analytics system

is required.

Example Tracked Events

Examples:

- Notification sent
- Notification delivered

- Notification opened
- Link clicked

Figure Reference

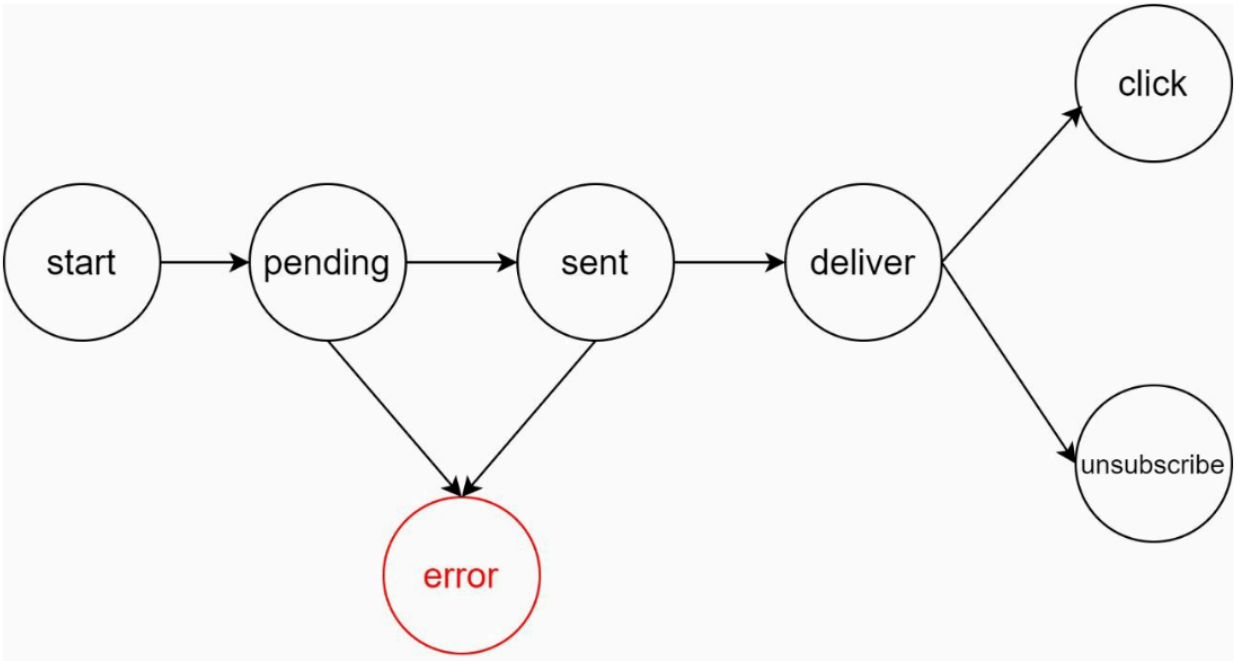

Figure shows:

- Example event tracking workflow for analytics

Benefits of Event Tracking

1. Understand User Behavior

Analyze how users interact with notifications.

2. Improve Notification Quality

Optimize:

- Timing
- Content
- Delivery strategy

3. Business Insights

Measure:

- Campaign effectiveness
- User engagement
- Conversion performance

Summary of Additional Components

Component	**Purpose**
Notification Template	Reusable notification structure
Notification Settings	User opt-in/opt-out control
Rate Limiting	Prevent notification overload
Retry Mechanism	Retry failed notifications
Push Notification Security	Authenticate verified clients

Queue Monitoring	Detect processing bottlenecks
Event Tracking	Collect analytics and engagement data